

UNIVERSITATEA TEHNICĂ A MOLDOVEI
FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ

Disciplina: Tehnici de programare
Lucrarea practică nr. 6.

Elaborat: Turcu Anastasia;
Grupa: SI-264;

Verificat: Reițman Patricia.

Chişinău, 2026

Introducere

În cadrul laboratorului „Informații invizibile: forensics și steganografie”, voi utiliza limbajul Python pentru a descoperi și analiza informații codate sau ascunse în fișiere. Voi lucra pe fișierele pregătite pentru laborator și pe propriile fișiere.

În prima parte, voi studia transformarea textului în octeți și în format hexazecimal. Voi realiza un decoder care recunoaște și decodează formatele URL, hex și Base64, inclusiv mesaje cu mai multe straturi de codare. Apoi voi determina cheia unui mesaj criptat prin XOR și voi căuta parolele corespunzătoare unor hash-uri, folosind o listă de cuvinte. Programul va primi date prin argumente introduse în terminal, va salva rezultatele într-un fișier JSON și va fi verificat prin teste.

În a doua parte, voi identifica tipul real al fișierelor după primii octeți, voi extrage secvențele de text și voi analiza metadatele unei fotografii, inclusiv coordonatele GPS. De asemenea, voi extrage o arhivă ascunsă într-o imagine și voi verifica fișierele din interiorul acesteia. La final, voi ascunde un mesaj în biții pixelilor unei imagini prin metoda LSB și îl voi recupera pentru a verifica funcționarea programului.

Voi prezenta codul, comenzile executate și rezultatele obținute în raport, iar fișierele lucrării vor fi încărcate în depozitul Git.

Capitolul 1-Transformarea textului în octeți și în format hexazecimal (A1)

```
(.venv) PS C:\Users\user\Desktop\lucrarenrutty> python -c "s='Salut'; b=s.encode(); print('Octeti:
, b); print('Primul octet:', b[0]); print('Hex:', b.hex()); print('Refacut:', bytes.fromhex(b.hex(
))"
Octeti: b'Salut'
Primul octet: 83
Hex: 53616c7574
Refacut: b'Salut'
(.venv) PS C:\Users\user\Desktop\lucrarenrutty>
```

fig.1-Programul transformă textul în octeți și în format hexazecimal, apoi reface octeții inițiali.

Capitolul 1.1 Verifică mesajul cu straturi

```
(.venv) PS C:\Users\user\Desktop\lucrarenrutty> python arsenal/forensics/dec
ode.py probe/mesaj.txt --fisier
{
  "strat": "url",
  "straturi": [
    {
      "strat": "url",
      "rezultat": "NDY0YzQxNDc3YjczNzQ3MjYxNzQ3NTcyNjk2YzY1NWY2MTc1NWY2NjZmN
zM3NDVmNjQ2NTYzNmY2NDYxNzQ2NTdk"
    },
    {
      "strat": "base64",
      "rezultat": "464c41477b73747261747572696c655f61755f666f73745f6465636f6
46174657d"
    },
    {
      "strat": "hex",
      "rezultat": "FLAG{straturile_au_fost_decodate}"
    }
  ],
  "rezultat": "FLAG{straturile_au_fost_decodate}",
  "rezultat_hex": "464c41477b73747261747572696c655f61755f666f73745f6465636f6
46174657d"
}
(.venv) PS C:\Users\user\Desktop\lucrarenrutty>
```

Fig.1.1-Programul citește fișierul mesaj.txt, decodează pe rând straturile URL, Base64 și hex și afișează mesajul ascuns: FLAG{straturile_au_fost_decodate}.

Capitolul 2. Verifică fișierul XOR

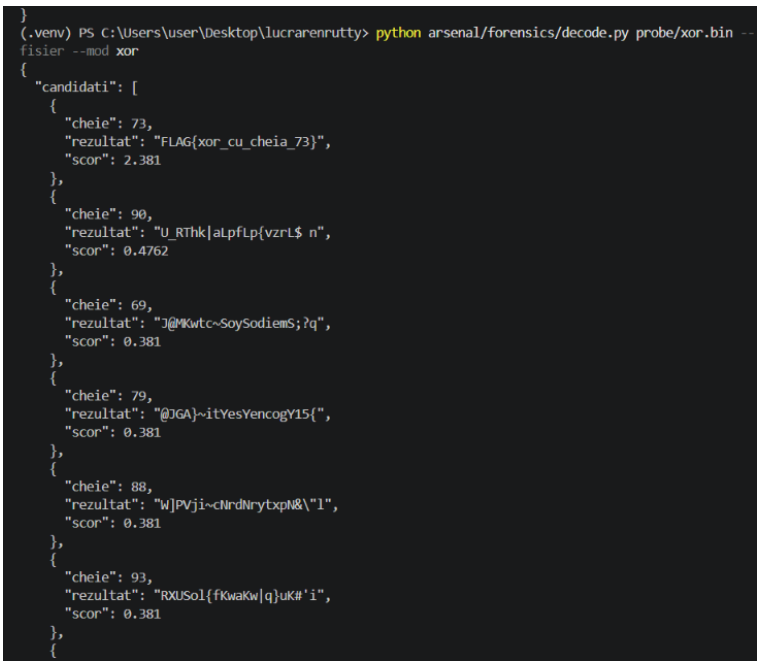


Fig.2.1- Programul verifică posibilele chei XOR și găsește cheia 73, cu care descifrează mesajul FLAG{xor_cu_cheia_73}.



Fig.2.2- Programul afișează alte chei încercate și rezultatele obținute, care nu reprezintă mesaje ușor de citit.

```
  "cheie": 84,
  "rezultat": "[Q\\ZferoB^hB~uxt|B*.`",
  "scor": 0.2381
},
{
  "cheie": 92,
  "rezultat": "SYTRnmzgJv`Jv}p|tJ\"&h",
  "scor": 0.2381
},
{
  "cheie": 72,
  "rezultat": "GM@Fzyns^bt^bidh`^62|",
  "scor": 0.1905
},
{
  "cheie": 74,
  "rezultat": "EOBDx{lq\\`v\\`kfjb\\40~",
  "scor": 0.1429
},
{
  "cheie": 80,
  "rezultat": "_UX^bavkFzlFzq|pxF.*d",
  "scor": 0.1429
},
{
  "cheie": 76,
  "rezultat": "CIDB~}jwZfpZfm`ldZ26x",
  "scor": 0.0952
},
{
  "cheie": 81,
  "rezultat": "^TY_c`wjG{mG{p}qyG/+e",
  "scor": 0.0952
},
{
  "cheie": 86,
  "rezultat": "YS^Xdgpm@|j@|wzv~@(,b",
```


Fig.2.3-Sunt afișate în continuare alte variante de chei și rezultatele lor, împreună cu scorul acordat fiecăreia.

Capitolul 3. Verificarea hash-urilor, găsirea parolelor și testarea programului

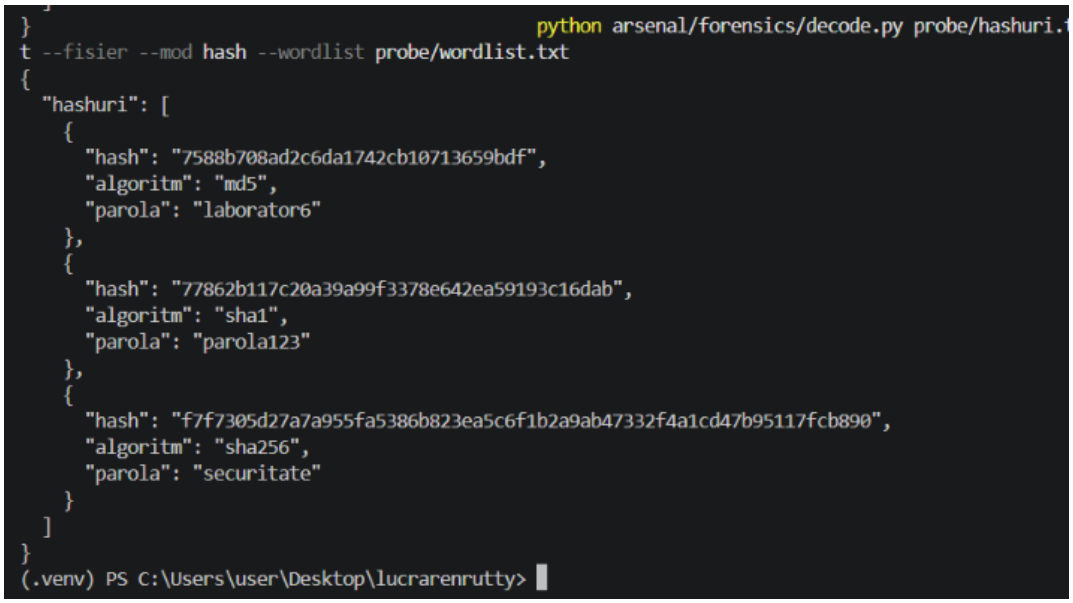


Fig.3.1-Programul identifică algoritmii MD5, SHA-1 și SHA-256 și găsește parolele corespunzătoare în lista de cuvinte: laborator6, parola123 și securitate.

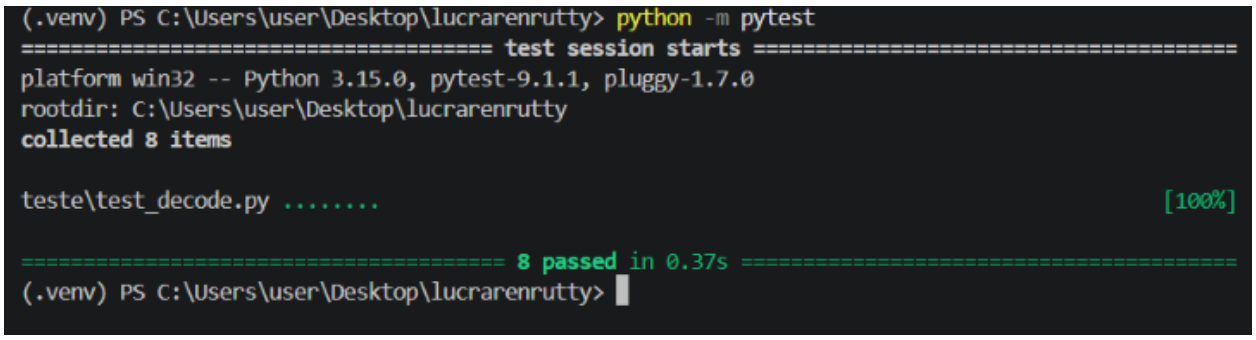


Fig.3.2-Rularea testelor cu ajutorul comenzii python -m pytest: toate cele 8 teste au trecut cu succes.

Capitolul 4 – Analiza fișierelor și extragerea arhivei ascunse.



Fig.4.1-Programul identifică tipurile reale ale fișierelor și extrage secvențele de text din imagine.



Fig.4.2-Programul afișează metadatele și coordonatele GPS și citește mesajul FLAG{arhiva_ascunsa_la_coada} din arhiva ascunsă.

Capitolul 5 – Ascunderea mesajului în imagine (B5)

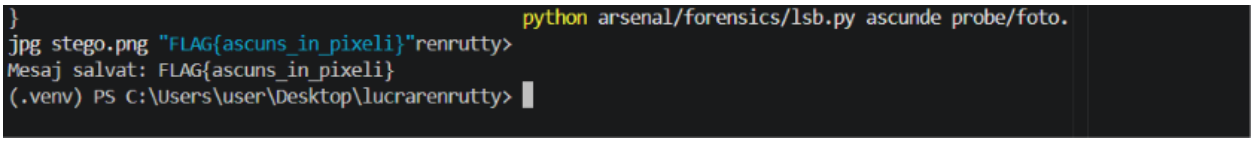


fig.5-Programul ascunde mesajul FLAG{ascuns_in_pixeli} în pixelii imaginii și salvează rezultatul în fișierul stego.png.

Capitolul 6 – Recuperarea mesajului ascuns (B6).

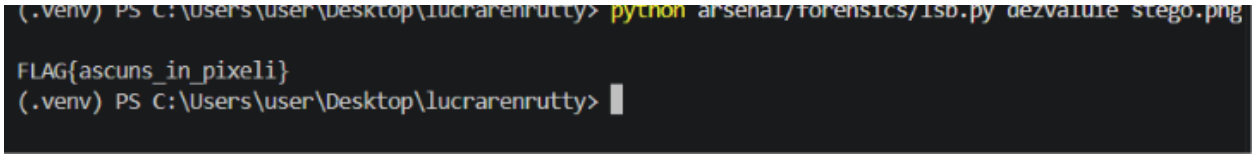


fig.6-Programul extrage din imaginea stego.png mesajul ascuns anterior: FLAG{ascuns_in_pixeli}.

Concluzie

În urma realizării acestui laborator, am învățat să folosesc Python pentru a descoperi informații codate și ascunse în fișiere. Am transformat textul în octeți și în format hexazecimal, am decodat mesaje cu straturi URL, Base64 și hex, am găsit cheia XOR și am identificat parolele corespunzătoare unor hash-uri folosind o listă de cuvinte.

De asemenea, am identificat tipurile reale ale fișierelor, am extras texte și metadate, inclusiv coordonate GPS, și am recuperat o arhivă ascunsă într-o imagine. Am ascuns un mesaj în pixelii unei imagini prin metoda LSB și l-am recuperat cu succes. Rezultatele au fost salvate în fișiere, iar toate cele 8 teste au trecut. Acest laborator m-a ajutat să înțeleg că un fișier poate conține informații care nu sunt vizibile la simpla deschidere și că verificarea programelor este necesară pentru obținerea unor rezultate corecte.